

MLS-SBO: Mathematical Formulation

Gradient-enhanced Moving-Least-Squares surrogate-based optimization
as implemented in `scp_uno/mls_sbo_core.py`

1 Problem and sample archive

$$\min_x f(x) \quad \text{s.t.} \quad c_j(x) \leq 0, \quad j = 1, \dots, m, \quad \ell \leq x \leq u. \quad (1)$$

Internally all quantities live in the normalized box $\xi \in [0, 1]^d$ with $\xi = (x - \ell)/(u - \ell)$; gradients are scaled accordingly (`MLSSBOptimizer._eval`). Every true evaluation performs one FEM solve plus adjoints and stores the full tuple in the archive

$$\mathcal{S} = \{(\xi_i, f_i, g_i, c_i, J_i)\}_{i=1}^N, \quad g_i = \nabla f(\xi_i) \in \mathbb{R}^d, \quad J_i = \nabla c(\xi_i) \in \mathbb{R}^{m \times d}. \quad (2)$$

2 Weights and length scale

All fits use Gaussian weights centred at an evaluation point z (`MovingLeastSquares._weights`):

$$w_i(z) = \exp\left(-\frac{\|z - \xi_i\|^2 - r_{\min}^2(z)}{2h^2}\right), \quad r_{\min}^2(z) = \min_l \|z - \xi_l\|^2. \quad (3)$$

Subtracting $r_{\min}^2$ (so the largest weight is always 1) is exact — a weighted least-squares solution is invariant to a common scaling of the weights — and prevents underflow at small h ; this is what makes the interpolation limit $h \rightarrow 0$ and linear exactness hold at *any* length scale.

Length-scale evolution. At iteration k , with trust-region radius δ_k and centre ξ_k (`_min_dist_in_tr`),

$$h_k = \text{clip}(\gamma_{\text{ls}} d_{\min}, h_{\min}, h_{\max}), \quad d_{\min} = \min_{\substack{i: \|\xi_i - \xi_k\|_{\infty} \leq \delta_k \\ \xi_i \neq \xi_k}} \|\xi_i - \xi_k\|, \quad (4)$$

i.e. h tracks the minimal distance from the trust-region centre to a sampled point inside the trust region ($\gamma_{\text{ls}} = \text{ls_factor}$, default 2). As the trust region shrinks and samples cluster, the fit localizes at the pace of the sampling. Only the window of the `max_points = 60` samples nearest the centre enters any fit (`_fit_surrogate`), the incumbent best always retained.

3 Per-query gradient-enhanced linear MLS (the surrogate)

Used by the batch acquisition (§7) and as the source of the planar model. Outputs are standardized per output o : $\tilde{y} = (y - \mu_o)/\sigma_o$. At a query point z the local linear model $q(s) = \alpha + \beta^\top s$, $s = \xi - z$, is fitted by weighted least squares to **both observation types** (Hermite MLS):

$$\begin{array}{ll} \text{value rows:} & \alpha + \beta^\top s_i \approx \tilde{f}_i, & \text{weight } w_i(z), \\ \text{gradient rows:} & \beta_l \approx \tilde{g}_{i,l}, \quad l = 1, \dots, d, & \text{weight } w_i(z). \end{array}$$

Stationarity gives one shared $(d+1) \times (d+1)$ normal system for all $m+1$ outputs (`_solve_local`; $\varepsilon = \text{Tikhonov jitter}$):

$$\begin{pmatrix} \sum_i w_i & \sum_i w_i s_i^\top \\ \sum_i w_i s_i & \sum_i w_i (s_i s_i^\top + I) \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \sum_i w_i \tilde{f}_i \\ \sum_i w_i (s_i \tilde{f}_i + \tilde{g}_i) \end{pmatrix}, \quad (5)$$

where the $+I$ block is the contribution of the gradient equations. The prediction mean is α ; the *diffuse derivative* is β (the true derivative adds weight-motion terms, see §5). Properties (test-enforced): exact reproduction of affine functions at any h ; interpolation of the nearest sample's value and gradient as $h \rightarrow 0$.

Uncertainty proxy. MLS has no native variance; the acquisition uses the sample-density complement with *unshifted* weights $\bar{w}_i(z) = e^{-\|z - \xi_i\|^2 / 2h^2}$:

$$\sigma(z) = \sqrt{\max(0, 1 - \sum_i \bar{w}_i(z))} \in [0, 1], \quad \nabla \sigma = -\frac{\nabla(\sum_i \bar{w}_i)}{2\sigma}. \quad (6)$$

4 Exploitation model: centre-frozen anchored separable quadratic

The step model freezes the weights at the trust-region centre ξ_k — the “moving” of MLS happens *across* iterations, as the centre moves — so the model is an exact polynomial whose values and gradients are mutually consistent for the SQP subproblem solver (`AnchoredSeparableQuadratic`).

Intermediate variables (per output o and coordinate j ; `intermediate`). With “linear”, $y_j = \xi_j$. With “mma”, a reciprocal transform is placed on the *descent* side of the anchor gradient $g_{k,j}^{(o)}$, with asymptote distance $a = \max(a_0, 1.3\delta_k)$:

$$y_j = \begin{cases} \frac{1}{\xi_j - L_j}, & g_{k,j}^{(o)} < 0, & L_j = \xi_{k,j} - a & \text{(lower asymptote),} \\ \frac{1}{U_j - \xi_j}, & g_{k,j}^{(o)} > 0, & U_j = \xi_{k,j} + a & \text{(upper asymptote),} \\ \xi_j, & g_{k,j}^{(o)} = 0. \end{cases} \quad (7)$$

Let $t_j(\xi) = y_j(\xi) - y_j(\xi_k)$ and $c_j = g_{k,j} / \frac{dy_j}{d\xi_j}(\xi_k)$. The model of output o is

$$\boxed{m_o(\xi) = f_{k,o} + \sum_{j=1}^d c_{j,o} t_j(\xi) + \frac{1}{2} \sum_{j=1}^d q_{j,o} t_j(\xi)^2} \quad \frac{\partial m_o}{\partial \xi_j} = (c_{j,o} + q_{j,o} t_j) \frac{dy_j}{d\xi_j}(\xi). \quad (8)$$

Since $t(\xi_k) = 0$, the anchor is exact by construction: $m_o(\xi_k) = f_{k,o}$ and $\nabla m_o(\xi_k) = g_k^{(o)}$ — the fully-linear-model requirement of trust-region theory, obtained for free because adjoint gradients are available.

Diagonal curvature fit. With $w_i = w_i(\xi_k)$ frozen and the gradient residuals expressed in y -space,

$$h_{i,j} = \left. \frac{\partial f}{\partial y_j} \right|_{\xi_i} - c_j = \frac{g_{i,j}}{\frac{dy_j}{d\xi_j}(\xi_i)} - c_j, \quad (9)$$

two observation types enter the weighted least squares for q :

$$\begin{aligned} \text{gradient rows } (d \text{ per sample, decoupled): } & t_{ij} q_j \approx h_{i,j}, & \text{weight } w_i, \\ \text{value rows } (1 \text{ per sample, coupled): } & \frac{1}{2} \sum_j q_j t_{ij}^2 \approx r_i, & \text{weight } w_i^v = \frac{4d w_i}{\|t_i\|^2}, \end{aligned}$$

where $r_i = y_i^{\text{obs}} - f_k - \sum_j c_j t_{ij}$ is the value residual. The value-row weight removes the units imbalance (gradient equations are $O(t)$, value equations $O(t^2)$: raw least squares lets gradients drown values at local distances) and gives one value equation the mass of the sample's d gradient equations. Value rows are enabled per output by `fit_values` $\in \{\text{none}, \text{constraints (default: feasibility is a statement about values), all}\}$. Gradient-only outputs use the decoupled quotient

$$q_j = \frac{\sum_i w_i t_{ij} h_{i,j}}{\sum_i w_i t_{ij}^2 + \varepsilon}; \quad (10)$$

value-carrying outputs solve the coupled $d \times d$ normal system (once per iteration, per output), with $v_i = t_i \odot t_i$:

$$\left[\text{diag}\left(\sum_i w_i t_{ij}^2\right) + \frac{1}{4} \sum_i w_i^v v_i v_i^\top + \varepsilon I \right] q = \sum_i w_i (t_i \odot h_i) + \frac{1}{2} \sum_i w_i^v r_i v_i. \quad (11)$$

On a separable quadratic the fit is exact (test-enforced). The bandwidth floor `min_fit_neighbors` (default 1 = tightest) can force h up to the k -th-nearest-sample distance.

Planar variant (`model="planar", PlanarMLSModel`). The per-query MLS of (5) is solved *once* at ξ_k with frozen weights, giving the affine model $m(\xi) = \hat{\alpha} + \hat{\beta}^\top (\xi - \xi_k)$ in raw units. Its value and slope blend neighbour values *and* gradients; it does not hard-interpolate the incumbent (fully-linear accuracy, which trust-region theory accepts) and carries no curvature term.

5 Why the model is centre-frozen (consistency)

For a *moving* fit the true derivative of the prediction is

$$\frac{d\alpha(z)}{dz} = \underbrace{\beta(z)}_{\text{diffuse}} + \underbrace{\frac{\partial\alpha}{\partial w} \frac{\partial w}{\partial z}}_{\text{weight motion}}, \quad (12)$$

so the diffuse part alone is *not* the derivative of the value actually returned. Handing an SQP solver the pair (moving value, diffuse gradient) gives it an inconsistent problem and corrupts its line searches — a defect measured on the GGP benchmark. Freezing w at ξ_k per iteration makes the model a polynomial, for which value and gradient are exactly consistent; the diffuse derivative remains only inside the heuristic batch acquisition (§7).

6 Subproblem, trust region, restarts

Step subproblem (`_solve_subproblem`); this is the whole step in the sequential regime `batch_size = 1`:

$$\xi^+ = \arg \min_{\xi} m_0(\xi) \quad \text{s.t.} \quad m_j(\xi) \leq 0, \quad \xi \in [\max(0, \xi_k - \delta), \min(1, \xi_k + \delta)], \quad (13)$$

solved by SLSQP with the models' analytic gradients, multistarted from $\{\xi_k, \xi_{\text{best}}\}$ plus two random trust-region points; fallbacks in order: phase-1 feasibility restoration $\min_{\xi} \sum_j \max(0, m_j(\xi))^2$, then L-BFGS-B on the penalized model merit. If $\xi^+ = \xi_k$ (model KKT point), a *null step* shrinks δ without spending a true evaluation.

Acceptance and radius. With the ℓ_1 merit $\varphi(\xi) = f(\xi) + \mu \sum_j \max(0, c_j(\xi))$ and its model counterpart m^φ , the ratio test is

$$\rho_k = \frac{\varphi(\xi_k) - \varphi(\xi^+)}{m^\varphi(\xi_k) - m^\varphi(\xi^+)}. \quad (14)$$

The centre moves when the actual reduction is positive and $\rho_k \geq \eta_a$; the radius follows the classical three-zone rule: shrink $\times 0.5$ if the step failed or $\rho_k < 0.25$; hold in the middle band; expand $\times 2$ (capped) if $\rho_k \geq 0.5$ and the step reached at least 0.8δ .

Restarts. On trust-region collapse ($\delta < \delta_{\min}$) or stall, restart from the incumbent best with radius cycling $\delta \leftarrow \delta_0 / 2^{(r-1) \bmod 3}$ and one randomized evaluation inside the new region (so the refit differs from the run that stalled), up to `n_resets` times: the full evaluation budget is always spent, as a sequential optimizer such as MMA would.

7 Batch mode (optional, $q = \text{batch_size} > 1$)

Point 1 is the subproblem solution above; points $2, \dots, q$ minimize the penalized lower-confidence-bound acquisition shared with GE-SBO (`gesbo_core.propose_batch`, duck-typed on the surrogate of (5)):

$$A_j(\xi) = \hat{m}_0(\xi) - \kappa_j \sigma(\xi) + \varrho \sum_c \max(0, \hat{m}_c(\xi) + \text{shift}_c)^2 + w_r \sum_{b < j} e^{-\|\xi - \xi_b\|^2 / 2h_r^2}, \quad (15)$$

with $\kappa_j = \kappa_0 \gamma_\kappa^{j-1}$ an exploration ladder, $\hat{m}$ the standardized per-query MLS means, σ the density proxy, and the last term a diversity repulsion from already-selected batch members.

8 Reproducibility

The optimizer is deterministic (seeded RNG, deterministic linear algebra). Bit-reproducibility of a full run additionally requires a deterministic FEM backend: the `amjax` (JAX/XLA) solver carries a per-process $O(10^{-13})$ perturbation from thread-pool reduction ordering, which the chaotic trust-region trajectory amplifies into *different local minima*; `fem_solver: direct` (scipy SuperLU) is verified bit-deterministic end-to-end.

9 Defaults

Symbol	Config field	Default
γ_{ls}	ls_factor	2.0
$h_{\min}, h_{\max}$	ls_min, ls_max	10^{-3} , 2.0
window N	max_points	60
ε	regularization	10^{-8}
intermediate variables	intermediate	"linear"
a_0	asy_init	0.5
value rows	fit_values	"constraints"
bandwidth floor	min_fit_neighbors	1
model	model	"quadratic"
$\delta_0, \delta_{\min}, \delta_{\max}$	tr_init, tr_min, tr_max	0.25, 10^{-5} , 0.75
shrink / expand	tr_shrink, tr_expand	0.5, 2.0
η_a , shrink zone, expand zone	eta_accept, eta_shrink, eta_expand	10^{-4} , 0.25, 0.5
μ	penalty	100
restarts	n_resets	8
q	batch_size	4; 1 = sequential regime
$\kappa_0, \gamma_\kappa, w_r$	kappa_base, kappa_growth, repulsion_weight	1, 2, 1